

TRABAJO PRACTICO INTEGRADOR

Programacion I

Informe Final

Agustin Pereyra

Soto Matias

1) Explicar en un informe teórico los conceptos aplicados: (relacionarlo con el proyecto)

•Listas:

Las listas son estructuras que permiten guardar varios elementos en una sola variable. Son ordenadas, mutables y pueden contener datos de distintos tipos. Son útiles para recorrer, filtrar u ordenar información.

Por qué lo usamos en nuestro integrador:

Porque necesitábamos almacenar y manejar la información de múltiples países de forma dinámica. Las listas nos permitieron agrupar todos los registros leídos desde el archivo CSV.

Cómo lo usamos:

Leyendo el csv, los países y columnas se guardan como listas en variables, para luego hacer acciones. Iteramos sobre esas listas para aplicar filtros, búsquedas y cálculos estadísticos.

Ejemplo:

Captura de pantalla:

```
with open("data/paises.csv", "r") as archivo:
    encabezado = archivo.readline().strip().split(",")
    lineas = archivo.readlines()
    filtrados = []

    for i in lineas:
        fila = i.strip().split(",")

        if any(filtro in elemento for elemento in fila):
            fila[2] = fila[2] + " km2" #Le agrego el km2
            filtrados.append(fila)
```

•Diccionarios:

Los diccionarios almacenan pares de datos *clave-valor*, lo que permite acceder rápidamente a un valor con su clave. Son ideales para representar objetos o registros con varios atributos.

Por qué lo usamos en nuestro integrador:

Porque cada país tiene varios datos (nombre, población, superficie y continente) que debían agruparse bajo un mismo registro.

Cómo lo usamos:

Cada país fue representado como un diccionario, por ejemplo:

```
{"nombre": "Argentina", "poblacion": 45376763, "superficie": 2780400, "continente": "América"}
```

Esto nos permitió acceder fácilmente a cada campo dentro del código.

Ejemplo:

Captura de pantalla:

```
with open("data/paises.csv", "r", newline="", encoding="utf-8") as archivo:
    datos = csv.DictReader(archivo)

    paises = list(datos)

    totalSuma = 0
    cantidad = 0
    for i in paises:
        if continente == "todos" or i["continente"] == continente:
            totalSuma += int(i["poblacion"])
            cantidad += 1

    promedioTodos = totalSuma / cantidad
```

• Funciones:

Las funciones son bloques de código reutilizables que ejecutan una tarea específica. Mejoran la organización y evitan repetir código.

Por qué lo usamos en nuestro integrador:

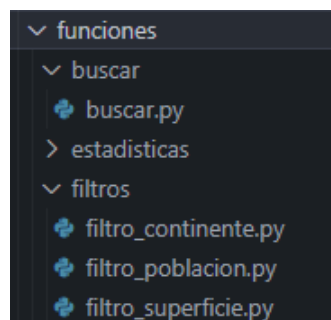
Para dividir el programa en módulos más claros: búsqueda, filtrado, ordenamiento, estadísticas, utils, etc. Cada función se encarga de una sola tarea.

Cómo lo usamos:

Creamos funciones como `get_paises()`, `buscar_pais()`, `filtrar_poblacion()`, `estadistica_poblacion()`, etc., para que el programa principal solo tenga que llamar a las funciones según la opción elegida por el usuario.

Ejemplo:

Captura de pantalla:



```
def buscar_pais(): etc . . .
```

•Condicionales:

Las estructuras `if`, `elif` y `else` permiten tomar decisiones dentro del programa dependiendo del cumplimiento de una condición.

Por qué la usamos en nuestro integrador:

Para validar opciones del menú, evitar errores y controlar el flujo del programa según las elecciones del usuario.

Cómo la usamos:

En el menú principal verificamos la opción ingresada por el usuario, y según su valor se ejecuta la función correspondiente o se muestra un mensaje de error.

Ejemplo:

Captura de pantalla:

```
if opcion == "1":
    estadistica_poblacion()
elif opcion == "2":
    estadistica_poblacionContinente()
elif opcion == "3":
    estadistica_superficie()
elif opcion == "4":
    estadistica_pais()
elif opcion == "5":
    break
else:
    print("Error, Intente Nuevamente con una opcion 1-5 ")
```

•Ordenamientos:

El ordenamiento consiste en organizar los datos de una lista según un criterio (alfabético, numérico, etc.). En Python se puede usar `sorted()` o el método `.sort()`.

Por qué lo usamos en nuestro integrador:

Porque necesitábamos mostrar los países ordenados por nombre, población o superficie, de forma ascendente o descendente.

Cómo lo usamos:

Aplicamos la función `sort()` y `sorted()` con una *lambda* como clave de ordenamiento para listar de mayor a menor población, nombre y superficie.

Ejemplo: Captura de pantalla

```
países_ordenados = sorted(países, key=lambda x: x['superficie'])
```

```
if orden.upper() == "A-Z":  
    países.sort()  
elif orden.upper() == "Z-A":  
    países.sort(reverse=True)
```

•Estadísticas básicas:

Las estadísticas básicas permiten obtener información descriptiva de los datos, como promedios, máximos o mínimos.

Por qué lo usamos en nuestro integrador:

Para calcular indicadores como el país con mayor y menor población, el promedio de población por continente y la cantidad total de países.

Cómo lo usamos:

Recorrimos la lista de países y aplicamos operaciones como `max()`, `min()` y sumatorias, combinadas con condicionales para filtrar por continente.

Ejemplo:

Captura de pantalla

```
países = list(datos)  
  
pais_mayor = max(países, key=lambda x: x["poblacion"])  
pais_menor = min(países, key=lambda x: x["poblacion"])
```

•Archivos CSV:

Los archivos CSV (Valores Separados por Comas) almacenan datos tabulares en texto plano y son muy usados para intercambio de información entre programas.

Por qué lo usamos en nuestro integrador:

Porque el dataset base del proyecto debía contener los datos de los países, y el formato CSV facilita su lectura desde Python.

Cómo lo usamos:

Usamos el módulo `csv` para abrir y leer el archivo. Cada fila se convirtió en un diccionario que luego fue agregado a la lista principal de países.

Ejemplo:

Captura de pantalla

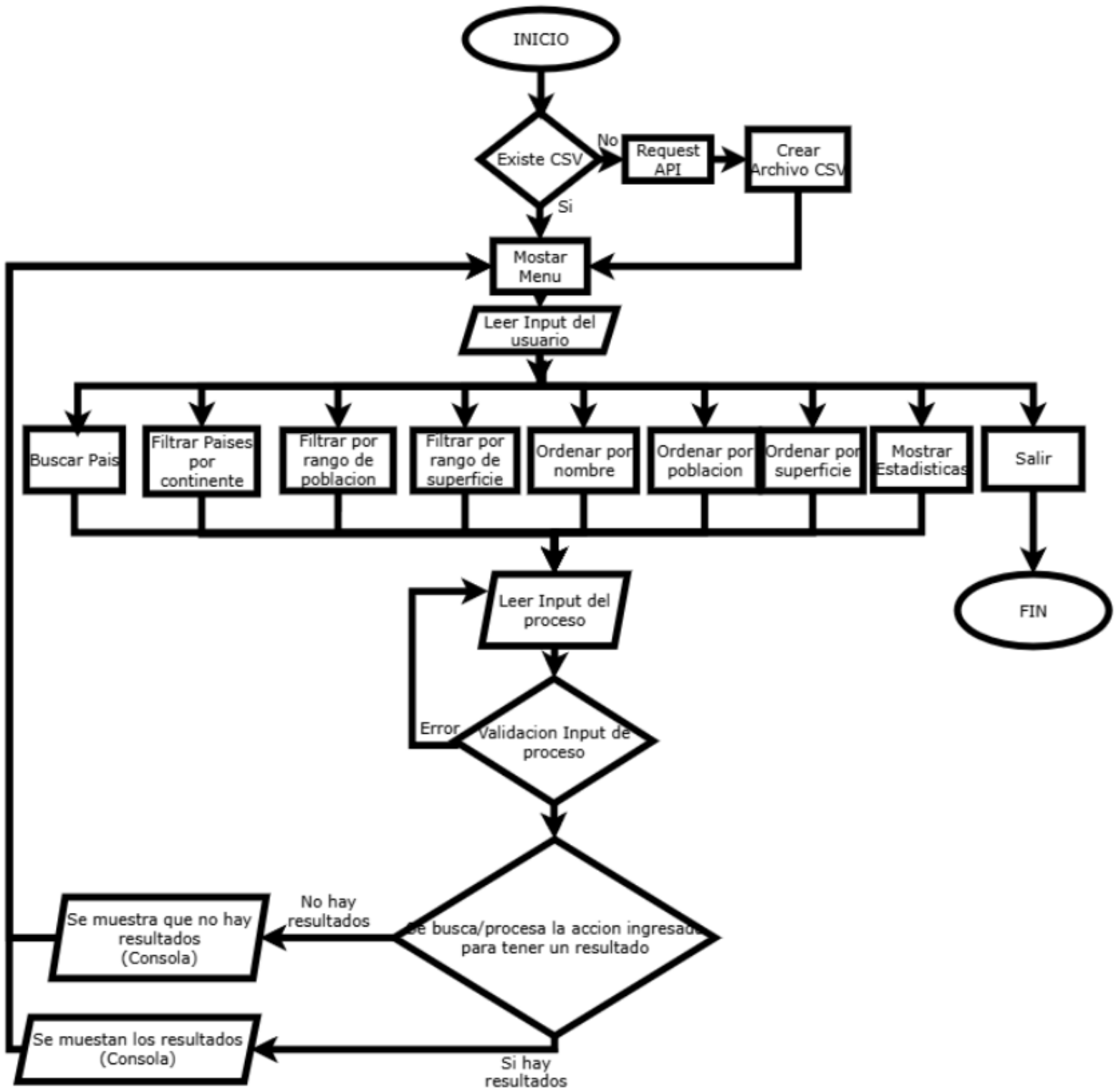
```
1 nombre,poblacion,superficie,continente
2 Comoros,919901,1862,Africa
3 Benin,13224860,112622,Africa
4 United States,340110988,9372610,North America
5 Mayotte,320901,374,Africa
```

```
with open("data/paises.csv", "r", newline="", encoding="utf-8") as archivo:
    datos = csv.DictReader(archivo)
```

Fuentes:

- o W3Schools. (s.f.). *Python Lists*. W3Schools. Recuperado el 20 de octubre de 2025, de https://www.w3schools.com/python/python_lists.asp
- o W3Schools. (s.f.). *Python Functions*. W3Schools. Recuperado el 20 de octubre de 2025, de https://www.w3schools.com/python/python_functions.asp
- o Python Software Foundation. (s.f.). *More control flow tools*. Python.org. Recuperado el 20 de octubre de 2025, de <https://docs.python.org/3/tutorial/controlflow.html>
- o W3Schools. (s.f.). *Python Dictionaries*. W3Schools. Recuperado el 20 de octubre de 2025, de https://www.w3schools.com/python/python_dictionaries.asp
- o Python Software Foundation. (s.f.). *CSV file reading and writing*. Python.org. Recuperado el 20 de octubre de 2025, de <https://docs.python.org/3/library/csv.html>
- o Python Software Foundation. (s.f.). *Sorting techniques*. Python.org. Recuperado el 20 de octubre de 2025, de <https://docs.python.org/3/howto/sorting.html>
- o Python Software Foundation. (s.f.). *statistics — Mathematical statistics functions*. Python.org. Recuperado el 20 de octubre de 2025, de <https://docs.python.org/3/library/statistics.html>

2) Definir el flujo de operaciones principales en un diagrama o esquema.



Librerías investigadas e implementadas:

#Consultamos tanto la documentación oficial como en otras paginas web.

Request: Es una librería integrada a Python que nos permitió hacer el llamado a la API de países para así procesar los datos necesarios (especificados en el endpoint), para luego guardarlos en el archivo CSV.

```
import requests # type: ignore

def get_paises():
    try:
        res = requests.get("https://restcountries.com/v3.1/all?fields=name,population,area,continents")
        res.raise_for_status()
        todo = res.json()
        csv = [
            "nombre,poblacion,superficie,continente\n"
        ]

        info = list(map(
            lambda e:
                # el pais Saint Helena, Ascension and Tristan da Cunha me daba mal formato en el csv para hacer
                f"{'Saint Helena (Tristan da Cunha)' if e['name']['common']=='Saint Helena, Ascension and Trista
                todo
            ))

        csv += info

        with open("data/paises.csv", "w") as archivo:
            archivo.writelines(csv)

    except requests.exceptions.RequestException as error:
        print(error)
    except Exception as error:
        print(error)
```

Rich: Es una librería externa a Python que nos permitió darle estilo a la consola de la terminal. Investigamos como colorear texto, hacer paneles con mensajes de éxito/error, crear una tabla con columnas y filas para mostrar la información, recuadrar información, y todo lo que tiene que ver con la parte visual.

Ejemplo:

```
from rich.console import Console # type: ignore
from rich.table import Table # type: ignore
from rich.panel import Panel # type: ignore
from rich.box import HEAVY # type: ignore
```

```
console = Console()
```

```
console.clear()
console.print("\n", Panel(f"[underline]ERROR:[/underline] {e}", style="bold red", box=HEAVY, title=":x: ERROR!"))
console.input("\nPresione Enter para continuar..")
```


Os: Es una librería integrada a Python que usamos para verificar la existencia del archivo CSV, nos sirvió para validar la existencia del archivo y así controlar el flujo del programa para que no rompa por si se borra antes o después o en el medio de la ejecución.

Ejemplo:

```
def asegurar_archivo(console):  
    if not os.path.exists("data/paises.csv"):  
        console.print("[yellow]El archivo 'paises.csv' no existe. Descargando datos desde la API...[/yellow]")  
        get_paises()
```

Conclusión grupal:

Como grupo, concluimos que este proyecto nos permitió integrar y aplicar los principales conceptos aprendidos durante la cursada de Programación 1. A través del desarrollo del sistema de países, pudimos trabajar de manera práctica con listas, diccionarios y funciones, comprendiendo cómo estructurar un programa modular y ordenado.

Además, aprendimos la importancia del control de flujo mediante condicionales y bucles, así como el uso de estructuras de datos adecuadas para filtrar, ordenar y analizar información. El trabajo con archivos CSV nos permitió entender cómo gestionar datos persistentes y validar su formato, reforzando la necesidad de aplicar controles de errores y mensajes claros al usuario.

Por último, la elaboración del diagrama de flujo y la organización del código nos ayudaron a visualizar el funcionamiento general del programa y a mejorar nuestras habilidades de trabajo colaborativo, comunicación y resolución de problemas dentro de un contexto de desarrollo real.